```
#!/usr/bin/env python3
"""
Bot Vinted : repère les annonces sous-évaluées et t'alerte sur Telegram et/ou Dis

Principe :
  1. Pour chaque recherche de config.json, on récupère les annonces les plus réce
  2. On estime le prix de revente = médiane des prix de la recherche x sell_throu
  3. Marge estimée = prix de revente - prix d'achat - frais d'envoi supposés.
  4. Si marge et ratio prix/estimation sont bons (et marque, état, mots-clés OK)

Usage :  pip install requests  &&  python vinted_bot.py
"""
import json
import os
import random
import sqlite3
import statistics
import sys
import time

import requests

HERE = os.path.dirname(os.path.abspath(__file__))
CFG = json.load(open(os.path.join(HERE, "config.json"), encoding="utf-8"))
BASE = CFG.get("domain", "https://www.vinted.fr")
CFG.setdefault("telegram", {})
CFG["telegram"]["bot_token"] = os.environ.get("TELEGRAM_BOT_TOKEN") or CFG["teleg
CFG["telegram"]["chat_id"] = os.environ.get("TELEGRAM_CHAT_ID") or CFG["telegram"
CFG["discord_webhook"] = os.environ.get("DISCORD_WEBHOOK") or CFG.get("discord_we
DB_PATH = os.path.join(HERE, "seen.db")

HEADERS = {
    "User-Agent": (
        "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 "
        "(KHTML, like Gecko) Chrome/124.0 Safari/537.36"
    ),
    "Accept": "application/json, text/plain, */*",
    "Accept-Language": "fr-FR,fr;q=0.9",
}

session = requests.Session()
session.headers.update(HEADERS)
```

```
# ---------- Base de données (annonces déjà vues) ----------
def init_db():
    db = sqlite3.connect(DB_PATH)
    db.execute("CREATE TABLE IF NOT EXISTS seen (id TEXT PRIMARY KEY)")
    db.commit()
    return db


def is_seen(db, item_id):
    return db.execute("SELECT 1 FROM seen WHERE id=?", (str(item_id),)).fetchone(


def mark_seen(db, item_id):
    db.execute("INSERT OR IGNORE INTO seen VALUES (?)", (str(item_id),))
    db.commit()


# ---------- Vinted ----------
def refresh_session():
    """Vinted exige un cookie de session : on le récupère via la page d'accueil."
    session.cookies.clear()
    session.get(BASE, timeout=20)


def fetch_items(search):
    params = {
        "search_text": search["text"],
        "order": "newest_first",
        "per_page": 96,
        "page": 1,
    }
    if search.get("max_price"):
        params["price_to"] = search["max_price"]
    for attempt in range(2):
        r = session.get(f"{BASE}/api/v2/catalog/items", params=params, timeout=20
        if r.status_code in (401, 403):
            refresh_session()
            continue
        r.raise_for_status()
        return r.json().get("items", [])
    return []


def get_price(item):
    p = item.get("price")
    if isinstance(p, dict):
        return float(p.get("amount", 0))
```

```
    return float(p or 0)


# ---------- Évaluation ----------
def evaluate(items, search):
    """Retourne la liste des bonnes affaires triées par marge décroissante."""
    brands_ok = [b.lower() for b in search.get("brands", CFG.get("brands", []))]
    bad_words = [w.lower() for w in CFG.get("blacklist_words", [])]
    conditions_ok = [c.lower() for c in CFG.get("conditions_ok", [])]

    def keep(it):
        title = (it.get("title") or "").lower()
        brand = (it.get("brand_title") or "").lower()
        status = (it.get("status") or "").lower()
        if brands_ok and not any(b in brand for b in brands_ok):
            return False
        if any(w in title for w in bad_words):
            return False
        if conditions_ok and not any(c in status for c in conditions_ok):
            return False
        return get_price(it) > 0

    pool = [it for it in items if keep(it)]
    if len(pool) < CFG.get("min_comparables", 8):
        return []

    median = statistics.median(get_price(it) for it in pool)
    resale = median * CFG.get("sell_through_factor", 0.85)
    ship = CFG.get("shipping_cost", 3.0)

    deals = []
    for it in pool:
        price = get_price(it)
        margin = resale - price - ship
        ratio = price / resale if resale else 1
        if margin >= CFG.get("min_margin", 8) and ratio <= CFG.get("max_ratio", 0
            deals.append({"item": it, "price": price, "resale": resale, "margin":
    return sorted(deals, key=lambda d: d["margin"], reverse=True)


# ---------- Notifications ----------
def format_message(d, search_name):
    it = d["item"]
    return (
        f"🔥 {it.get('title')}\n"
        f"Marque : {it.get('brand_title')} | Taille : {it.get('size_title')} | Ét
        f"Prix : {d['price']:.2f}€ | Revente estimée : {d['resale']:.0f}€ | "
```

```
        f"Marge estimée : {d['margin']:.0f}€\n"
        f"Recherche : {search_name}\n"
        f"{it.get('url') or BASE + '/items/' + str(it.get('id'))}"
    )


def notify(text):
    tg = CFG.get("telegram", {})
    if tg.get("bot_token") and tg.get("chat_id"):
        requests.post(
            f"https://api.telegram.org/bot{tg['bot_token']}/sendMessage",
            data={"chat_id": tg["chat_id"], "text": text},
            timeout=20,
        )
    dc = CFG.get("discord_webhook")
    if dc:
        requests.post(dc, json={"content": text}, timeout=20)


# ---------- Boucle principale ----------
def run_pass(db, first_run):
    for search in CFG["searches"]:
        try:
            items = fetch_items(search)
            for d in evaluate(items, search):
                item_id = d["item"]["id"]
                if is_seen(db, item_id):
                    continue
                mark_seen(db, item_id)
                if not first_run:
                    notify(format_message(d, search["text"]))
                    print("Alerte :", d["item"].get("title"))
            if first_run:
                for it in items:
                    mark_seen(db, it["id"])
        except Exception as e:  # on ne plante jamais le passage
            print(f"[{search['text']}] erreur : {e}", file=sys.stderr)
        time.sleep(random.uniform(2, 5))


def main():
    once = "--once" in sys.argv  # un seul passage (pour GitHub Actions)
    db = init_db()
    first_run = db.execute("SELECT COUNT(*) FROM seen").fetchone()[0] == 0
    refresh_session()
    if first_run:
        print("Premier passage : les annonces existantes sont ignorées.")
```

```
    run_pass(db, first_run)
    while not once:
        time.sleep(CFG.get("interval_seconds", 90) + random.uniform(0, 20))
        run_pass(db, False)


if __name__ == "__main__":
    main()
```